Tunzuri System Architecture

1 November 2026

1. Purpose

Tunzuri is a cloud hospital management system for private hospitals, clinics and faith-based facilities. One installation serves many facilities (tenants). Each facility runs its whole patient journey in it: registration, queues, triage, consultation, laboratory, imaging, pharmacy, wards, maternity, theatre, cashier, insurance, inventory and Ministry of Health reports.

This document describes how the system is built: its components, how a request moves through them, how facilities are kept apart, how people sign in, and how the clinical, billing and stock workflows fit together. The product scope and the user manual cover what each screen does for staff.

Three rules shape most of the design:

1. **Every facility sees only its own data.** Isolation is enforced in one place, for every query, rather than in each screen.
2. **Every service rendered is billed.** Orders create their own charges, so nothing depends on someone remembering to bill.
3. **Every stock movement goes through one ledger.** Stock on hand is always the sum of recorded movements, each traced to a delivery, a patient or a department.

2. Technology stack

Layer	Technology
Language and framework	PHP 8.3, Laravel 12 (streamlined Laravel 11+ structure)
Database	PostgreSQL in development and production; SQLite in memory for tests
Sessions, cache, queue	Database drivers (`database` sessions, cache and queue)
Front end	Server-rendered Blade views, Tailwind CSS 4, Alpine.js 3, Boxicons, built with Vite 7
Authentication scaffolding	Laravel Breeze, heavily extended for per-facility sign-in
Testing	Pest 3 with the Laravel plugin
Code style	Laravel Pint
Mail	Laravel mail; `log` mailer locally, SMTP in production
SMS	`SmsGateway` contract, currently bound to a log-only gateway
Analytics	Self-hosted page-view tracker, production only, with patient-data safeguards

There is no separate API or single-page app. Every screen is a Blade page with small Alpine components for interactive parts such as searchable dropdowns and the facility-code lookup.

3. System overview



The application is a single Laravel deployment. Controllers validate input and call **services** in `app/Services`, which hold the business rules (billing, dispensing, stock, claims and so on). Services work through Eloquent models. Slow outbound work, such as bursts of email and every SMS, runs on the database queue.

4. Code layout

Path	Contents
`app/Http/Controllers`	One controller per area, plus `Admin/` (facility administration), `Platform/` (platform operator) and `Auth/` (sign-in, resets, sign-up)
`app/Http/Middleware`	`SetFacilityContext`, `EnsurePermission`, `EnsureFacilityOnboarded`, `SecurityHeaders`
`app/Services`	Domain services: `EncounterService`, `BillingService`, `CashierService`, `DispensingService`, `StockLedger`, `ProcurementService`, `ClaimService`, `HmisReport`, `FacilityProvisioner` and others
`app/Models`	About 55 Eloquent models; `Concerns/BelongsToFacility` and `Concerns/RecordsAudit` are shared behaviour

Path	Contents
`app/Authorization`	`RolePermissions`: the permission names, role defaults and assignable work areas
`app/Enums`	Workflow states and code lists: `VisitStatus`, `QueueStage`, `StockReason`, `Role`, `Payer`, `BillingType` and others
`app/Support`	Cross-cutting helpers: `FacilityContext`, `Money`, `Navigation`, `MailPressure`
`app/Notifications`, `app/Jobs`	Emails (invitations, password resets, verification, facility ready) and queued jobs
`database/migrations`	One migration per table, plus later alterations
`database/seeders`	ICD-10 codes, a demo hospital, and a second facility with people who work at both
`resources/views`	Blade pages, grouped by area, and shared components in `components/`
`tests/Feature`	Pest feature tests for each area

5. Multi-tenancy

5.1 Model

Tunzuri uses a **shared database with row-level isolation**. Every tenant-owned table has a `facility_id` column. One PostgreSQL database holds all facilities. This keeps provisioning instant, makes one migration run apply to everyone, and allows the platform-wide totals the operator dashboard needs.

5.2 How isolation is enforced

Isolation lives in one trait, `App\Models\Concerns\BelongsToFacility`, used by every tenant model:

- **Global scope.** Every query on the model adds `where facility_id = <current facility>`.
- **Automatic stamping.** New records get the current facility's id when none is given.
- **Fail-closed for operators.** For a platform operator the context carries a "hide tenant data" flag, and the scope adds `where 0 = 1`, so an operator never loads patient records by accident. Platform screens that need cross-facility counts opt out deliberately with `withoutGlobalScope('facility')` and only ever aggregate.

The current facility is held in `App\Support\FacilityContext`, a per-request static set by the `SetFacilityContext` middleware and cleared when the request ends.

5.3 Ordering guarantee

Route model binding must run *after* the facility is known. Otherwise a URL like `/visits/123` could load another facility's visit before the scope applies. `bootstrap/app.php` puts `SetFacilityContext` ahead of `SubstituteBindings` in the middleware priority list. A visit from another facility therefore returns 404, not someone else's record.

5.4 Shared reference data

Some data is global by design: the ICD-10 code list (`icd10_codes`), facilities themselves, users (people) and the number-sequence table, which is keyed by facility. Everything clinical, financial and stock-related is per facility.

5.5 Known gap

Outside a web request (queued jobs, scheduled commands, `tinker`) no facility is set. The scope then applies **no** filter. Current jobs only load single records by id, so nothing leaks today. Any future report or reminder job must set the context explicitly. Making the scope block by default, and adding PostgreSQL row-level security as a database backstop, are the recommended next steps.

6. Identity, sign-in and access

6.1 People and memberships

Table	Holds
`users`	The person: name, email (unique platform-wide), phone, global active flag, `is_platform_admin`. Only platform operators keep a password here.
`memberships`	The person at one facility: role, permissions, department, active flag, **that facility's password**, last sign-in there, and invitation and password-reset tokens.

One email can belong to several facilities through several memberships. The `User` model exposes `facility_id`, `role`, `facility`, `permissions` and `department`. These come from the membership in use for the current request, so the rest of the code reads them as if each user had one facility. The accessors are deliberately uncached, because the membership is set during the request.

6.2 Signing in

```
Sign-in page
  Facility code  [SMH ]  -> "St. Mary's Community Hospital, Kampala" (live lookup)
  Email          [....]
  Password       [....]  -> checked against THAT facility's membership only
```

- **Facility code first.** Codes are unique platform-wide. As the code is typed, the page shows the facility's name and district, using a rate-limited lookup of public facility names, so a person who works at several facilities sees where they are going.
- **One password per facility.** The password is checked against the membership at that facility. A password for one facility never opens another.
- **No in-app switching.** A session works in exactly one facility. Reaching another means signing out and signing in with its code. If the session's membership stops being usable (switched off, or the facility deactivated), the person is signed out.

- **No code for administrators and operators.** A facility administrator belongs to exactly one facility, so they can leave the code blank and sign in with email and password. Platform operators also leave it blank and use their account password. Staff always need the code.
- **No enumeration.** A wrong code, an unknown email and a wrong password all return the same error. A miss still runs a password-hash check, so it takes as long as a wrong password. Attempts are rate-limited per email and IP.
- **Remembered code.** The facility code is stored in a cookie on the device. "Remember me" logins are not used, because a restored login would have no facility.

6.3 Roles and permissions

`App\Authorization\RolePermissions` defines the permission names (for example `patients.view`, `cashier.manage`, `inventory.approve`, `platform.manage`) and the work areas an administrator can tick: reception, clinical, laboratory, imaging, pharmacy, stores and reports.

- **Administrator:** every facility permission.
- **Staff:** only the permissions stored on their membership, from the areas ticked.
- **Platform administrator:** only `platform.manage`.

Each permission is registered as a Gate in `AppServiceProvider`. Routes are grouped behind `permission:<name>` middleware (`EnsurePermission`), and the sidebar (`App\Support\Navigation`) shows only items the person may open. Separation of duties is built in: stock approvals and waiver approvals stay with the administrator, apart from the people who count stock and take cash.

6.4 Adding staff: invitations and the privacy rule

A facility administrator must never learn that a person has an account at another facility. Adding staff therefore always works by invitation:

1. The administrator enters a name, an email and the access to give.
2. If the email is already a member **here**, the request is refused with that person's name. This is the only case that differs.
3. In every other case the response is identical: "Invitation sent", an *Invited* row showing the name the administrator typed, the same audit entries and the same email. That holds whether the email is new to Tunzuri or already used elsewhere. An account for a new person is created without an audit entry, so the trail does not differ.
4. The invitee opens the emailed link, chooses a password for this facility (a new person also gives their name), and is signed in there. Links last 7 days and only a SHA-256 hash of the token is stored.

Administrators belong to one facility only. An administrator cannot accept an invitation to another facility, and someone who works elsewhere cannot accept an administrator invitation. In both cases only the invitee sees why; the inviting administrator still sees an ordinary pending invitation. Promoting an existing member who works elsewhere is refused. Platform operators are told the reason directly.

Administrators manage access and status at their own facility only. Name, email and password belong to the person. "Last signed in" counts only sign-ins at that facility. Platform operators are exempt from the rule and can link an existing account directly.

6.5 Passwords, sessions and audit

- **Resets are per facility.** "Forgot password" takes a facility code and email (administrators can leave the code blank), emails a 60-minute link, and changes only that facility's password. The reply never reveals whether the details matched.
- **Profile and confirmation.** The current-password rule and the password-confirmation screen check the password of the facility in use, through `User::getAuthPassword()`.
- **Idle timeout.** The front end signs a person out after `HMS_IDLE_TIMEOUT` minutes without activity (15 by default).
- **Audit trail.** `RecordsAudit` logs creates, updates and deletes for audited models, with user, facility, IP address and user agent. Passwords and tokens are stripped from logged changes.

7. Request lifecycle

```
HTTP request
  -> StartSession, CSRF check, auth (web middleware group)
  -> SetFacilityContext
       * signs out inactive accounts
       * finds the membership chosen at sign-in (session key membership_id)
       * signs out if it is no longer usable
       * sets FacilityContext and shares facility and navigation with views
  -> SubstituteBindings   (route models load inside the facility scope)
  -> EnsurePermission     (permission:<name> on route groups)
  -> EnsureFacilityOnboarded (self-registered facilities finish setup first)
  -> Controller -> Service -> Models
  -> SecurityHeaders on the response
  -> SetFacilityContext::terminate clears the context
```

8. Domain modules

Module	Controllers	Main services	Main tables
Patients and visits	`PatientController`, `VisitController`	`PatientRegistrar`, `EncounterService`	`patients`, `visits`, `triage_records`, `consultations`, `diagnoses`
Appointments and queues	`AppointmentController`, `QueueController`	`QueueBoard`, `SmsNotifier`	`appointments`, `queue_entries`, `sms_messages`
Laboratory	`LabController`	`EncounterService`	`lab_tests`, `lab_requests`, `lab_request_items`
Radiology	`RadiologyController`	`EncounterService`	`imaging_studies`, `imaging_orders`

Module	Controllers	Main services	Main tables
Pharmacy	`PharmacyController`	`DispensingService`, `StockLedger`	`prescriptions`, `prescription_items`, `dispensations`
Wards	`WardController`, `Admin\WardController`	`WardManager`, `WardStockService`	`wards`, `beds`, `admissions`, `nursing_notes`, `medication_administrations`, `ward_consumptions`
Maternity and theatre	`MaternityController`, `TheatreController`	n/a	`antenatal_visits`, `deliveries`, `theatre_bookings`, `operation_notes`
Billing and cashier	`CashierController`, `ReceiptController`	`BillingService`, `CashierService`, `PriceBook`	`bills`, `bill_lines`, `payments`, `cashier_shifts`, `waivers`
Insurance	`InsuranceController`	`ClaimService`	`insurance_schemes`, `service_prices`, `item_prices`, `preauthorisations`, `claims`, `claim_lines`, `remittances`
Inventory	`InventoryController`, `StockController`	`StockLedger`, `ProcurementService`, `ItemCatalogue`, `StockReport`	`items`, `stock_locations`, `stock_batches`, `stock_movements`, `suppliers`, `purchase_orders`, `goods_receipts`, `requisitions`, `stock_adjustments`
Reports	`ReportController`, `DashboardController`	`HmisReport`	reads across the above
Facility administration	`Admin\*`	`FacilityProvisioner`	`departments`, `services`, `memberships`, `audit_logs`
Platform	`Platform\*`	`FacilityProvisioner`	`facilities` and cross-facility counts

9. Core workflows

9.1 Patient journey

A visit moves through a fixed state machine (`VisitStatus`), and `Visit::moveTo()` refuses illegal moves:

```
registered -> triaged -> consulting -> closed
     |           |           |
     +-----------+-----------+--> admitted -> discharged -> closed
```

Alongside the status, the visit appears in department queues (`QueueStage`: triage, consultation, laboratory, radiology, pharmacy, cashier) managed by `QueueBoard`. Each stage is tied to the permission that works it, so a lab technician's queue shows only laboratory work. Patient and visit numbers come from

`SequenceGenerator`, a per-facility counter incremented under a row lock so concurrent registrations never collide.

9.2 Charge capture

Billing is a side effect of clinical work, not a separate step:

1. An order is placed: a consultation, lab test, imaging study, dispensed medicine or ward item.
2. `PriceBook` picks the price. The insurance scheme's negotiated price applies if the visit has a scheme and one is set; otherwise the cash price.
3. `BillingService::charge()` adds a bill line tied to its source record. A unique source key makes charging idempotent: the same order can never be billed twice.
4. For insurance visits, `assertCanCharge()` blocks charges when the scheme requires preauthorisation and none is approved, or when the charge would pass the scheme's cover limit.
5. Cash visits with something owed are queued at the cashier.

9.3 Cashier, waivers and release

`CashierService` runs cashier shifts (opening float, expected and counted cash, variance) and takes payments by method. Waivers are requested by front-office staff and approved or rejected by an administrator. For cash patients, `DispensingService::releaseIfSettled()` hands medicines over only once the bill is settled. Insurance and corporate visits are released straight away.

9.4 Insurance claims

`ClaimService` compiles a claim for a scheme and period from that scheme's bill lines, numbers it, moves it from draft to submitted, and matches remittances against it. A claim records the total claimed and the amount remitted.

9.5 Stock ledger

`StockLedger` is the only code that changes stock:

- **Batches.** Stock is held in batches per store (`stock_batches`: item, location, batch number, expiry, unit cost, quantity).
- **First expiry, first out.** `consume()` draws from batches with the nearest expiry date first, then batches without one, locking the rows it changes.
- **Every change is a movement.** Each one is written to `stock_movements` with a reason (`StockReason`: purchase, transfer in or out, dispense, ward issue, internal use, lab use, return, adjustment, opening, donation), the patient or department, and the user.
- **Balances.** Stock on hand is the sum of movements, so balances, stock cards and usage reports always agree.

```
Supplier
   |  goods received (batch, expiry, unit cost)
   v
Main store
   |  requisition and issue
   v
Pharmacy  |  Ward stores  |  Laboratory  |  Theatre
   |  dispense, ward issue, lab use, internal use
   v
stock_movements: one ledger for every store
(item, batch, location, quantity, reason, patient or department, user)
```

`ProcurementService` handles purchase orders, goods received (creating batches) and requisitions from department stores. A requisition the main store cannot fully cover stays open. Adjustments need administrator approval. `StockReport` builds stock levels, reorder suggestions, stock cards and usage by department, employee or item.

9.6 Facility onboarding

A facility arrives in one of two ways:

- **Self sign-up.** The owner registers, verifies their email, then completes a three-step wizard: facility details and collection currency, wards and beds, go live. `EnsureFacilityOnboarded` holds them in the wizard until it is finished.
- **Provisioned by a platform operator.** It goes live straight away.

`FacilityProvisioner` creates the facility, its standard departments, stores, two wards with beds, starter prices and the first administrator's membership, all in one transaction.

10. Money and currency

- **Whole units.** Amounts are stored as unsigned integers in the currency's whole units. There are no cents, which suits shilling and franc currencies.
- **Per-facility currency.** Each facility chooses its collection currency (80 supported, East African first). `App\Support\Money` formats amounts in the current facility's currency.
- **Bills keep their currency.** Each bill stores the currency it was raised in, so a later change does not relabel history. Changing currency does not convert prices.
- **Limit:** currencies that need decimals (USD, EUR, KWD) can be selected but can only charge whole amounts until amounts move to minor units.

11. Background work, mail and SMS

- **Queue.** Laravel's `database` queue, worked by `php artisan queue:work`. `composer run dev` starts it locally.

- **Mail pressure.** `User::notify()` sends mail during the request until a burst passes `HMS_MAIL_QUEUE_AFTER` messages (5 by default), then queues the rest (`SendNotification`), so a burst cannot stall a request.
- **SMS.** `SmsNotifier` records each message in `sms_messages` and dispatches `SendSms`. The job calls the `SmsGateway` contract, bound to `LogSmsGateway`, which writes to the log. A real provider plugs in by binding another implementation in `AppServiceProvider`.
- **Emails sent:** email verification, facility ready, staff invitations, per-facility password resets, and operator password resets. Facility emails use "<facility> via Tunzuri" as the sender name.

12. Files

Patient photos and radiology images are stored on the private `local` disk (`patient-photos/`, `imaging/`). They are never served from a public path: controllers stream them after the same facility-scoped lookup and permission check as the record they belong to.

13. Reporting

- **Facility dashboard:** today's activity and revenue.
- **Reports:** cash collected, department activity and the Ministry of Health HMIS 105 (outpatient attendance by sex and age band, diagnoses) and HMIS 108 (inpatient) monthly reports, built by `HmisReport` from daily records.
- **Stock reports:** levels, reorder suggestions, stock cards and usage.
- **Platform overview and facility detail:** aggregate counts only, such as facilities by status, sign-ups per week, visits, patients, occupancy, collections, staff activity, setup and stock health. No patient record is loaded.

14. Front end

- **Pages.** Server-rendered Blade with two layouts: `app` for signed-in pages, and `guest` for sign-in, sign-up, invitations and the setup wizard.
- **Shared components:** `search-select` (a searchable dropdown with hidden search terms), `facility-code-field` (live facility lookup), modals, inputs, the item and quantity fields.
- **Styling.** Tailwind CSS 4 with brand tokens, plus Boxicons for icons.
- **Interactivity.** Alpine.js components are registered in `resources/js/app.js`: `searchSelect`, `facilityLookup`, sample-account fill, the idle timeout and print.
- **Build.** Vite compiles assets to `public/build`. `npm run dev` serves them live.
- **Analytics.** The page-view tracker (`layouts/analytics.blade.php`) loads only in production. It never runs on pages whose address holds a secret token (password reset, email verification, invitations). It also blanks internal referrers first, because search text in them could name a patient.

15. Security summary

Concern	Measure
Cross-facility access	Global facility scope on every tenant model; bindings resolved inside the scope; operators blocked from tenant rows
Credentials	One password per facility; bcrypt hashing; uniform sign-in errors with timing equalised; rate limits on sign-in, lookups and invitation acceptance
Account discovery	Invitations, resets and sign-in never reveal whether an email exists elsewhere
Tokens	Invitation and reset tokens stored only as SHA-256 hashes, with expiry
Sessions	One facility per session; signed out when access ends; idle timeout; regenerated at sign-in
Transport and headers	HTTPS forced in production; `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`, `Permissions-Policy`
Accountability	Audit log of changes with user, facility, IP and user agent
Third parties	Analytics excluded from token pages; internal referrers removed

16. Configuration

Setting	Purpose
`DB_*`	PostgreSQL connection
`SESSION_DRIVER`, `CACHE_STORE`, `QUEUE_CONNECTION`	`database` by default
`MAIL_MAILER` and SMTP settings	`log` locally; set SMTP for real delivery
`HMS_IDLE_TIMEOUT`	Minutes of inactivity before sign-out (default 15)
`HMS_MAIL_QUEUE_AFTER`	Emails sent in-request before queuing (default 5)
`HMS_DEMO_PASSWORD`	Password of the sample accounts listed on the local sign-in page
`config/hms.php`	The settings above, plus the default currency (`UGX`)

17. Testing

- **Tool and database.** Pest feature tests run against SQLite in memory (`phpunit.xml`), with array session and cache and the sync queue.
- **Helpers.** `tests/Pest.php` provides `provisionHospital()`, `openCashVisit()` and `membershipOf()`. `Tests\TestCase::actingAs()` opens the user's only facility, as signing in with its code would.
- **Factories.** `UserFactory` accepts `facility_id`, `role` and `permissions` and turns them into a membership, so tests read naturally.

- **Coverage.** About 160 tests cover the workflows, tenant isolation, the privacy rule, per-facility sign-in and resets, the platform screens and the seeders. Run them with `php artisan test --compact`.

18. Operations

Task	Command
First setup	`composer run setup`
Local development	`composer run dev` (server, queue worker, logs, Vite)
Migrations	`php artisan migrate`
Demo data (rerunnable)	`php artisan db:seed` (ICD-10, St. Mary's, Gulu Family Health Clinic and people who work at both)
Create a platform operator	`php artisan hms:create-platform-admin`
Queue worker	`php artisan queue:work`
Format code	`vendor/bin/pint --dirty`

Production needs PHP 8.3 with `pdo_pgsql`, PostgreSQL, a running queue worker, SMTP mail, HTTPS, and backups of the database and the `storage/app` disk.

19. Known limitations and next steps

1. **Fail-open tenant scope outside web requests.** Make the scope block by default and add PostgreSQL row-level security (section 5.5).
2. **Whole-unit amounts.** Move to minor units before facilities collect in currencies with cents.
3. **Real SMS delivery.** Bind a provider to `SmsGateway`.
4. **Public sign-up** still reports "email already taken", which reveals that an account exists, and an existing person cannot start a second facility under the same email.
5. **No field-level encryption** of clinical free text. Rely on encrypted database storage at the host, and consider encrypting the most sensitive notes.